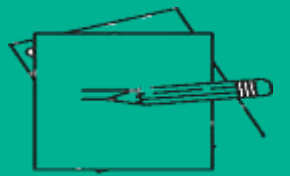
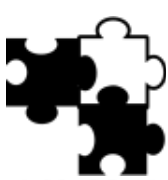


Taller de construcción de SW móvil


Semana 6 y 7



SABERES
PREVIOS



¿Qué efectos ha observado en la interfaz de usuario de una aplicación móvil?

¿Cómo producir efectos en la interfaz de usuario de una aplicación móvil?

Logro de la sesión



Al finalizar este bloque el alumno explica el uso de Hooks en react native, el ciclo de vida en componentes funcionales y la integración de hooks en el desarrollo de una app móvil, mediante el desarrollo de código que integre los conceptos.



Introducción a los Hooks en React Native

- Funciones que permiten usar estado y ciclo de vida en componentes funcionales.
- Evitan el uso de clases, simplificando el desarrollo
- Ejemplos clave: `useState` para manejar estado, `useEffect` para manejar efectos secundarios y ciclo de vida.

Ejemplo 1: Contador simple con `useState`, Al presionar el botón, `setCount` actualiza el estado y el texto se actualiza mostrando el nuevo valor.

```
import React, { useState } from 'react';
import { View, Button, Text } from 'react-native';

export default function Contador() {
  const [count, setCount] = useState(0);

  return (
    <View>
      <Text>Contador: {count}</Text>
      <Button title="Incrementar" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

Características clave:

- El estado declarado con `useState` es local al componente.
- React asegura que el estado persista incluso tras re-renderizados.
- `useState` puede ser llamado múltiples veces para manejar diferentes variables de estado en un mismo componente.
- Actualizar estado con la función `setX` es asíncronico y React agrupa actualizaciones para optimizar el renderizado.

Ciclo de vida tradicional vs Hooks

El ciclo de vida tradicional de un componente con métodos como `componentDidMount`, `componentDidUpdate` y `componentWillUnmount` se unifica en el hook `useEffect`. En componentes funcionales, `useEffect` reemplaza estos métodos.

`useEffect` permite ejecutar código cuando el componente se monta, actualiza o desmonta según las dependencias que se pasen.

Ejemplo sencillo: temporizador que se inicia al montar y se limpia al desmontar.

```
import React, { useState, useEffect } from 'react';
import { View, Text } from 'react-native';

export default function Temporizador() {
  const [segundos, setSegundos] = useState(0);

  useEffect(() => {
    const intervalo = setInterval(() => {
      setSegundos(s => s + 1);
    }, 1000);

    return () => clearInterval(intervalo); // Limpieza al desmontar
  }, []);

  return (
    <View>
      <Text>Segundos transcurridos: {segundos}</Text>
    </View>
  );
}
```

Resumen

Este componente crea un contador que incrementa un número cada segundo, utilizando:

- `useState` para manejar el estado local `segundos`.
- `useEffect` para iniciar un temporizador al montar y limpiar ese temporizador al desmontar.
- La UI se actualiza dinámicamente mostrando el tiempo transcurrido.

- `export default function Temporizador() {`

Define un componente funcional llamado `Temporizador` que será exportado por defecto para su uso en cualquier parte de la app.

- `const [segundos, setSegundos] = useState(0);`

Declara una variable de estado `segundos` inicializada en `0` y una función `setSegundos` para actualizar ese estado. El estado mantiene los segundos transcurridos.

- `useEffect(() => { ... }, []);`

El hook `useEffect` recibe una función que se ejecuta justo después del primer renderizado del componente. El arreglo vacío `[]` indica que este efecto solo se ejecuta al montar el componente y se limpia cuando se desmonta.

- `const intervalo = setInterval(() => { setSegundos(s => s + 1); }, 1000);`

Dentro del efecto, se crea un intervalo que cada 1000 ms (1 segundo) ejecuta la función que actualiza el estado `segundos`, sumando 1 al valor anterior `s`.

- `return () => clearInterval(intervalo);`

La función que retorna `useEffect` es una función de limpieza que se ejecuta al desmontar el componente. Aquí se limpia el intervalo evitando que siga ejecutándose después de que el componente desaparezca, previniendo fugas de memoria.

- `return ( <View> <Text>Segundos transcurridos: {segundos}</Text> </View> );`

La función `Temporizador` retorna la UI del componente: un contenedor `View` con un texto que muestra los segundos transcurridos. La variable `segundos` se actualiza cada segundo y React vuelve a renderizar para mostrar el nuevo valor.

Integración de hooks en app móviles

Uso de hooks para controlar estado, manejar navegación y efectos en la UI móvil.

Se muestra ejemplos con `useEffect` en React Native para cada caso clave del ciclo de vida de un componente funcional: cuando se monta, cuando se actualiza y cuando se desmonta.

Código que se ejecuta solo cuando el componente se monta, es decir, una vez. (equivalente a `componentDidMount`):

```
import React, { useEffect, useState } from "react";
import { View, Text, Button } from "react-native";

export default function EjemploMontaje() {
  // Estado local para mostrar mensaje de confirmación
  const [mensaje, setMensaje] = useState("");

  useEffect(() => {
    console.log("Componente montado");
    // Acción: actualizar mensaje al montar el componente
    setMensaje("El componente se montó correctamente");

    // Como el arreglo de dependencias está vacío [],
    // este efecto solo se ejecuta una vez al montar el componente
  }, []);

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Este efecto se ejecuta solo al montar</Text>
      <Text style={{ marginTop: 20, fontWeight: "bold" }}>{mensaje}</Text>
      <Button title="Limpiar mensaje" onPress={() => setMensaje("")} />
    </View>
  );
}
```

Explicación:

- Se agregó un estado `mensaje` para mostrar en la pantalla que el efecto se ejecutó.
- Dentro del `useEffect`, se escribe un log en consola y se actualiza el estado `mensaje`, lo que solo ocurre una vez cuando el componente se monta.
- El arreglo vacío `[]` como segundo argumento garantiza que el efecto no se ejecute en actualizaciones posteriores, solo en el montaje inicial.
- La UI muestra el mensaje actualizado y un botón para limpiar ese mensaje, demostrando la interacción posterior sin volver a disparar el efecto.

Integración de hooks en app móviles



Se muestra un ejemplo para cada caso clave del ciclo de vida de un componente funcional: Cuando el componente se actualiza

Código que se ejecuta cada vez que cambia el estado `count`, simulando el comportamiento de `componentDidUpdate` en componentes funcionales:

```
import React, { useState, useEffect } from "react";
import { View, Text, Button } from "react-native";

export default function EjemploActualizacion() {
  // Declaración del estado "count" inicializado en 0
  const [count, setCount] = useState(0);

  useEffect(() => {
    // Este efecto se ejecuta cada vez que cambia "count"
    // Aquí se puede poner cualquier código que deba reaccionar a cambios en el estado
    console.log(`El contador cambió a ${count}`);

    // Por ejemplo, actualizar otro estado, realizar llamadas a APIs, o efectos secundarios
    // En este caso solo se imprime en consola para evidenciar la actualización.

    // La dependencia "count" hace que este efecto se dispare cuando cambia el estado
  }, [count]);

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text style={{ fontSize: 24, marginBottom: 20 }}>Contador: {count}</Text>

      {/* Al presionar este botón se actualiza el estado "count" */}
      <Button title="Incrementar" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

Explicación de la ejecución:

- Se declara un estado `count` — número que comienza en 0.
- `useEffect` define un efecto que se ejecuta cada vez que cambia `count` (porque está en el array de dependencias `[count]`).
- Cada cambio imprime en la consola el nuevo valor de `count`.
- Esto simula el comportamiento de `componentDidUpdate` en clases, pero en un componente funcional.
- La UI muestra el contador y un botón para incrementarlo, activando la actualización y disparando el efecto al cambiar el estado.

Integración de hooks en app móviles

Se muestra un ejemplo para cada caso clave del ciclo de vida de un componente funcional: Cuando el componente se desmonta

Código que se ejecuta cuando el componente se monta y cuando se desmonta, se agrega un componente padre que controla si el componente EjemploDesmontaje está visible o no. Esto permite probar explícitamente el desmontaje ocultando el componente desde el padre con un botón:

```
1 import React, { useEffect, useState } from "react";
2 import { View, Text, Button } from "react-native";
3 import { SafeAreaView } from "react-native-safe-area-context";
4 function EjemploDesmontaje() {
5   const [mensaje, setMensaje] = useState("Componente no montado");
6
7   useEffect(() => {
8     console.log("Componente montado");
9     setMensaje("Componente montado");
10
11     // Función que se ejecutará al desmontar el componente
12     return () => {
13       console.log("Componente desmontado");
14       alert("El componente se desmontó");
15       setMensaje("Componente desmontado");
16       // Aquí se puede hacer limpieza, cancelar timers, listeners, etc.
17     };
18   }, []); // Solo se ejecuta al montar y limpia al desmontar
19   return (
20     <View style={{ flex: 1, justifyContent: "center", alignItems: "center", padding: 20 }}>
21       <Text style={{ fontSize: 18, marginBottom: 20 }}>Este efecto limpia al desmontar</Text>
22       <Text style={{ fontSize: 20, fontWeight: "bold", marginBottom: 20 }}>{mensaje}</Text>
23       <Text style={{ fontStyle: "italic" }}>
24         Para que se ejecute la función de limpieza, debe desmontarse este componente (por ejemplo, dejar de renderizarlo).
25       </Text>
26     </View>
27   );
28 }
29 export default function Contenedor() {
30   // Estado que controla si el componente EjemploDesmontaje se muestra o no
31   const [mostrarComponente, setMostrarComponente] = useState(true);
32
33   return (
34     <SafeAreaView style={{ flex: 1 }}>
35       <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
36         <Button
37           title={mostrarComponente ? "Ocultar Componente" : "Mostrar Componente"}
38           onPress={() => setMostrarComponente(!mostrarComponente)}
39         />
40
41         {/* Renderizaremos EjemploDesmontaje solo si mostrarComponente es true */}
42         {mostrarComponente && <EjemploDesmontaje />}
43       </View>
44     </SafeAreaView>
45   );
46 }
47
```

- Inicie la app con el componente `Contenedor`.
- El componente `EjemploDesmontaje` se muestra inicialmente y muestra "Componente montado".
- Al presionar el botón "Ocultar Componente", el componente se desmonta, ejecutando la función de limpieza: se muestra alerta, mensaje en consola y se actualiza el mensaje.
- Al volver a presionar "Mostrar Componente" este vuelve a montar y se repite el ciclo.

Esta es la forma práctica y habitual para probar desmontaje y limpieza con hooks `useEffect` en React Native controlando la presencia del componente desde un padre.

Ejemplo integrador

Ejemplo que incluye instrucciones y una gestión más completa del ciclo de vida usando `useEffect`, mostrando efectos visibles (mensajes en pantalla y logs) cuando el componente se monta, se actualiza (contador) y se desmonta

```
import React, { useState, useEffect } from "react";
import { View, Text, Button, StyleSheet, Alert } from "react-native";

export default function App() {
  // Estado para el contador
  const [count, setCount] = useState(0);
  // Estado para el temporizador
  const [seconds, setSeconds] = useState(0);
  // Estado para mostrar mensajes del ciclo de vida
  const [mensaje, setMensaje] = useState("");

  // useEffect para controlar ciclo de vida del temporizador (montaje y desmontaje)
  useEffect(() => {
    setMensaje("El componente se montó y el temporizador comenzó");
    console.log("Componente montado - temporizador iniciado");

    const interval = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    // Limpieza al desmontar
    return () => {
      clearInterval(interval);
      setMensaje("El componente se desmontó y el temporizador se detuvo");
      console.log("Componente desmontado - temporizador detenido");
    };
  }, []); // Solo se ejecuta al montar y desmontar

  // useEffect para detectar actualizaciones del contador (count)
  useEffect(() => {
    if (count === 0) return; // Evitar acción inicial al montar
    setMensaje(`El contador cambió a ${count}`);
    console.log(`El contador cambió a ${count}`);
  }, [count]); // Se ejecuta cada vez que "count" cambia
```

```
return (
  <View style={styles.container}>
    <Text style={styles.title}>Ejemplo React Native con Hooks</Text>

    {mensaje ? (
      <Text style={[styles.label, { color: "blue", marginBottom: 15 }]}>{mensaje}</Text>
    ) : null}

    <View style={styles.section}>
      <Text style={styles.label}>Contador:</Text>
      <Text style={styles.value}>{count}</Text>
      <Button title="Incrementar" onPress={() => setCount(count + 1)} />
    </View>

    <View style={styles.section}>
      <Text style={styles.label}>Temporizador {segundos}</Text>
      <Text style={styles.value}>{seconds}</Text>
    </View>
  </View>
);
```

- Usa `useState` para manejar el estado local tanto del contador como del temporizador.
- Usa `useEffect` para simular el ciclo de vida: el temporizador empieza a contar al montar y se detiene (limpia el intervalo) al desmontar.
- La UI muestra el contador con un botón para incrementar y un temporizador que actualiza cada segundo.

Se tiene el estado mensaje para reflejar en pantalla eventos del ciclo de vida y actualizaciones.

El primer `useEffect` gestiona montaje y desmontaje del temporizador:

Muestra mensaje y log al montar.

Limpia el intervalo, muestra mensaje y log al desmontar.

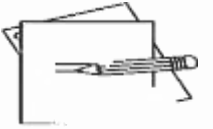
El segundo `useEffect` detecta cuando count cambia (excepto la inicialización) para mostrar mensaje y log indicando cada actualización.

Los mensajes se muestran arriba del contador, resaltando el estado actual respecto al ciclo de vida y actualizaciones al usuario.

```
// Estilos básicos para la app
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    padding: 20,
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
    fontWeight: "bold",
  },
  section: {
    marginBottom: 30,
    alignItems: "center",
    width: "80%",
  },
  label: {
    fontSize: 18,
    marginBottom: 10,
  },
  value: {
    fontSize: 32,
    marginBottom: 10,
    fontWeight: "bold",
  },
});
```

¿Preguntas?





Hooks

- Hooks permiten usar estado y ciclo de vida en componentes funcionales, evitando el uso de clases.
- Los hooks más importantes explicados son:
- `useState`: para manejar el estado local en componentes funcionales.
- `useEffect`: para manejar efectos secundarios y simular el ciclo de vida (montaje, actualización, desmontaje).

Ejemplos mostrados

- Contador simple con `useState`: Muestra cómo declarar un estado local con `useState` y actualizarlo con la función `setCount`.
- Temporizador con `useEffect`: Uso del hook `useEffect` para simular el ciclo de vida tradicional. Se inicia un temporizador en el montaje del componente y se limpia (detiene) cuando el componente se desmonta. Demuestra el manejo correcto de recursos y limpieza para evitar fugas de memoria.

- Ejecución solo al montar (`componentDidMount`): Código dentro de `useEffect` configurado con arreglo vacío para que solo se ejecute una vez al montar el componente.
- Ejecución al actualizar (`componentDidUpdate`): `useEffect` configurado para ejecutarse cada vez que cambia una variable de estado específica (`count`), para reaccionar a actualizaciones.
- Ejecución al desmontar (`componentWillUnmount`): Función de limpieza dentro de `useEffect` que se ejecuta al desmontar el componente, ideal para cancelar timers, listeners, etc.
- Se presentó un ejemplo integrador que combina `useState` y varios `useEffect` para manejar un contador, temporizador y mensajes visuales que reflejan eventos del ciclo de vida.

Logro de la sesión



Al finalizar este bloque el alumno explica los tipos de navegación entre pantallas de una app móvil en React Native, mediante la utilización de componentes Stack, Tab, Drawer.



Navegación entre pantallas en React Native

- La navegación es esencial en aplicaciones móviles para mover al usuario entre pantallas.
- React Navigation es la librería estándar para gestionar navegación en React Native.
- Permite usar diferentes tipos: Stack (pila), Tab (pestañas) y Drawer (menú lateral).

Configuración del entorno para navegar entre pantallas:

1. Crear el proyecto en blanco con Expo

En la terminal, ejecutar:

```
bash
npx create-expo-app MiAppNavegacion
cd MiAppNavegacion
```

Seleccionar la plantilla "blank" o "blank (TypeScript)" según preferencia.

2. Instalar dependencias para navegación

En el proyecto recién creado, instalar estas dependencias:

```
bash
npm install @react-navigation/native @react-navigation/native-stack @react-
navigation/bottom-tabs @react-navigation/drawer
npx expo install react-native-gesture-handler react-native-reanimated react-
native-screens react-native-safe-area-context
```

- `@react-navigation/native` es el módulo base de React Navigation.
- `@react-navigation/native-stack` para navegación stack.
- `@react-navigation/bottom-tabs` para pestañas.
- `@react-navigation/drawer` para drawer lateral.
- `react-native-gesture-handler` para manejo de gestos.
- `react-native-reanimated` para animaciones nativas.
- `react-native-screens` optimización de pantallas.
- `react-native-safe-area-context` para áreas seguras en pantallas.

3. Configurar `babel.config.js`

Editar el archivo `babel.config.js` para incluir el plugin `react-native-reanimated`:

```
js
module.exports = function(api) {
  api.cache(true);
  return {
    presets: ['babel-preset-expo'],
    plugins: ['react-native-reanimated/plugin'],
  };
};
```


Navegación Stack

Navegación parecida a una pila donde se puede avanzar y retroceder.

Se usa para flujos lineales, ejemplo: desde Home a Detalles.



Ejemplo básico de navegación Stack, se usó npx expo start para ejecutar

```
// 1. Importar react-native-gesture-handler como primer import para soporte de gestos nativos
import 'react-native-gesture-handler';

// 2. Importar React y componentes básicos
import React from 'react';
import { Button, Text, View } from 'react-native';

// 3. Importar contenedor base de navegación
import { NavigationContainer } from '@react-navigation/native';

// 4. Importar la función para crear el Stack Navigator
import { createNativeStackNavigator } from '@react-navigation/native-stack';

// 5. Crear la instancia del Stack Navigator
const Stack = createNativeStackNavigator();
```

- `createNativeStackNavigator`: crea un stack para controlar pantalla apilada.
- `navigation.navigate('Details')`: mueve a pantalla Details.
- `Stack.Screen`: registra cada pantalla con nombre y componente.

```
// 6. Definir la pantalla Home con botón que navega a Details
function HomeScreen({ navigation }) {
  return (
    <View style={{flex: 1, justifyContent: 'center', alignItems: 'center'}}>
      <Button title="Ir a detalles" onPress={() =>
        navigation.navigate('Details')} />
    </View>
  );
}
```

- Componente funcional que representa la pantalla "Home".
- `navigation` es un objeto pasado por React Navigation para cambiar de pantalla.
- Contiene un botón que al pulsarse navega a la pantalla llamada 'Details'.

```
// 7. Definir la pantalla Details con texto simple
function DetailsScreen() {
  return (
    <View style={{flex: 1, justifyContent: 'center', alignItems: 'center'}}>
      <Text>Pantalla Detalles</Text>
    </View>
  );
}
```

- Componente pantalla "Details" que solo muestra un texto.

```
// 8. Definir el Stack Navigator con las pantallas Home y Details
function StackNavigator() {
  return (
    <Stack.Navigator>
      <Stack.Screen name="Home" component={HomeScreen} />
      <Stack.Screen name="Details" component={DetailsScreen} />
    </Stack.Navigator>
  );
}
```

- Componente que define el stack con dos pantallas declaradas: Home y Details.
- `<Stack.Navigator>` Es el contenedor del stack.
- `<Stack.Screen>` registra cada pantalla con nombre y componente.

```
// 9. Componente principal App que envuelve la navegación en NavigationContainer
export default function App() {
  return (
    <NavigationContainer>
      <StackNavigator />
    </NavigationContainer>
  );
}
```

Navegación Tab

- createBottomTabNavigator crea barra con pestañas.
- Navegación horizontal para secciones clave.
- Cada pestaña es una ruta registrada en el Tab.Navigator.



Barra de pestañas inferior para navegar entre secciones principales.



Cada pestaña puede tener su propio stack.

Ejemplo básico de navegación Tab, se usó npx expo start para ejecutar

```
// 1. Importar react-native-gesture-handler para soportar gestos nativos
import 'react-native-gesture-handler';

// 2. Importar React y componentes básicos
import React from 'react';
import { Text, View } from 'react-native';

// 3. Importar contenedor de navegación y creadores de Tabs
import { NavigationContainer } from '@react-navigation/native';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

// 4. Crear instancia del Tab Navigator
const Tab = createBottomTabNavigator();

// 5. Componente Settings simple
function SettingsScreen() {
  return (
    <View style={{flex: 1, justifyContent: 'center', alignItems: 'center'}}>
      <Text>Configuraciones</Text>
    </View>
  );
}
```

```
// 6. Componente Profile simple
function ProfileScreen() {
  return (
    <View style={{flex: 1, justifyContent: 'center', alignItems: 'center'}}>
      <Text>Perfil</Text>
    </View>
  );
}

// 7. Definir Tab Navigator con pantallas Settings y Profile
function TabNavigator() {
  return (
    <Tab.Navigator>
      <Tab.Screen name="Settings" component={SettingsScreen} />
      <Tab.Screen name="Profile" component={ProfileScreen} />
    </Tab.Navigator>
  );
}

// 8. Componente raíz que envuelve toda la navegación en NavigationContainer
export default function App() {
  return (
    <NavigationContainer>
      <TabNavigator />
    </NavigationContainer>
  );
}
```

• Pantalla simple para pestaña "Profile".

- Define el tab navigator con dos pestañas.
- Cada pestaña tiene un nombre y un componente asociado.



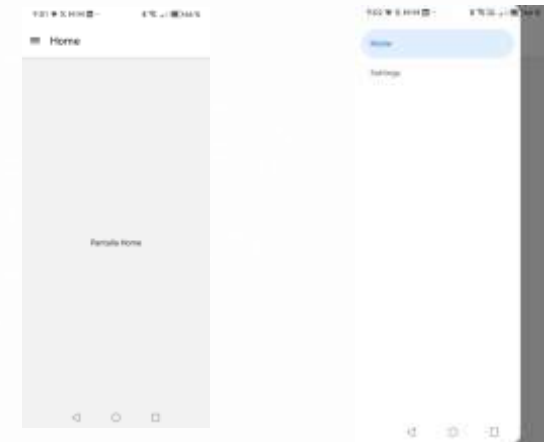
Navegación Drawer



Menú lateral desplegable con opciones.



Puede contener stacks o tabs. Facilita la navegación global entre secciones



Ejemplo básico de navegación Drawer, se usó npx expo start para ejecutar

```
// 1. Importar react-native-gesture-handler para soporte de gestos nativos
import 'react-native-gesture-handler';

// 2. Importar React y componentes básicos
import React from 'react';
import { Text, View } from 'react-native';

// 3. Importar NavigationContainer y Drawer Navigator
import { NavigationContainer } from '@react-navigation/native';
import { createDrawerNavigator } from '@react-navigation/drawer';

// 4. Crear instancia del Drawer Navigator
const Drawer = createDrawerNavigator();

// 5. Crear pantallas de ejemplo simples
function HomeScreen() {
  return (
    <View style={{flex: 1, justifyContent: 'center', alignItems: 'center'}}>
      <Text>Pantalla Home</Text>
    </View>
  );
}
```

```
function SettingsScreen() {
  return (
    <View style={{flex: 1, justifyContent: 'center', alignItems: 'center'}}>
      <Text>Pantalla Configuraciones</Text>
    </View>
  );
}

// 6. Definir el Drawer Navigator incluyendo las pantallas
function DrawerNavigator() {
  return (
    <Drawer.Navigator initialRouteName="Home">
      <Drawer.Screen name="Home" component={HomeScreen} />
      <Drawer.Screen name="Settings" component={SettingsScreen} />
    </Drawer.Navigator>
  );
}

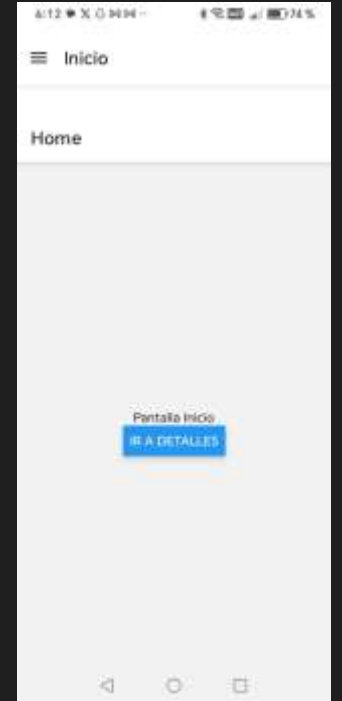
// 7. Componente raíz que envuelve la navegación en NavigationContainer
export default function App() {
  return (
    <NavigationContainer>
      <DrawerNavigator />
    </NavigationContainer>
  );
}
```

Ejemplo integrador

Ejemplo de una app móvil básica en React Native que integra la navegación entre pantallas utilizando Stack, Tab, Drawer.

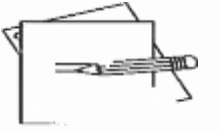
```
1 import 'react-native-gesture-handler'; // Manejo de gestos
2 import React from 'react';
3 import { Button, Text, View } from 'react-native';
4 import { NavigationContainer } from '@react-navigation/native';
5 import { createNativeStackNavigator } from '@react-navigation/native-stack';
6 import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
7 import { createDrawerNavigator } from '@react-navigation/drawer';
8 const Stack = createNativeStackNavigator();
9 const Tab = createBottomTabNavigator();
10 const Drawer = createDrawerNavigator();
11 // Pantalla Home con botón para ir a Details
12 function HomeScreen({ navigation }) {
13   return (
14     <View style={{ flex:1, justifyContent:'center', alignItems:'center' }}>
15       <Text>Pantalla Inicio</Text>
16       <Button title="Ir a Detalles" onPress={() => navigation.navigate('Details')} />
17     </View>
18   );
19 }
20 // Pantalla Detalles con botón para volver atrás
21 function DetailsScreen({ navigation }) {
22   return (
23     <View style={{ flex:1, justifyContent:'center', alignItems:'center' }}>
24       <Text>Pantalla Detalles</Text>
25       <Button title="Volver" onPress={() => navigation.goBack()} />
26     </View>
27   );
28 }
29 // Pantalla Configuraciones para Tab Navigator
30 function SettingsScreen() {
31   return (
32     <View style={{ flex:1, justifyContent:'center', alignItems:'center' }}>
33       <Text>Configuraciones</Text>
34     </View>
35   );
36 }
```

```
37 // Pantalla Perfil para Tab y Drawer
38 function ProfileScreen() {
39   return (
40     <View style={{ flex:1, justifyContent:'center', alignItems:'center' }}>
41       <Text>Perfil</Text>
42     </View>
43   );
44 }
45 // Stack Navigator con Home y Details
46 function StackNavigator() {
47   return (
48     <Stack.Navigator initialRouteName="Home">
49       <Stack.Screen name="Home" component={HomeScreen} />
50       <Stack.Screen name="Details" component={DetailsScreen} />
51     </Stack.Navigator>
52   );
53 }
54 // Tab Navigator con Settings y Profile
55 function TabNavigator() {
56   return (
57     <Tab.Navigator>
58       <Tab.Screen name="Settings" component={SettingsScreen} />
59       <Tab.Screen name="Profile" component={ProfileScreen} />
60     </Tab.Navigator>
61   );
62 }
63 // App principal con Drawer Navigator que incluye Stack y Tabs
64 export default function App() {
65   return (
66     <NavigationContainer>
67       <Drawer.Navigator initialRouteName="Stack">
68         <Drawer.Screen name="Stack" component={StackNavigator} options={{ title: 'Inicio' }} />
69         <Drawer.Screen name="Tabs" component={TabNavigator} options={{ title: 'Usuario' }} />
70         <Drawer.Screen name="Profile" component={ProfileScreen} options={{ title: 'Perfil Directo' }} />
71       </Drawer.Navigator>
72     </NavigationContainer>
73   );
74 }
```



¿Preguntas?





Tipos de navegación

- Stack Navigation (Pila): Permite avanzar y retroceder entre pantallas de forma lineal, como una pila de páginas. Es útil para flujos donde se navega de una pantalla principal a detalles y se puede regresar.
- Tab Navigation (Pestañas): Usa una barra inferior para cambiar entre secciones principales de la app. Cada pestaña representa una ruta y puede contener su propio stack.
- Drawer Navigation (Menú Lateral): Un menú lateral desplegable que facilita la navegación global entre secciones, y puede contener stacks o tabs.

Configuración Básica

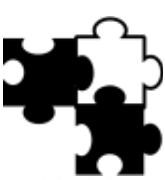
- Crear un proyecto con Expo.
- Instalar las dependencias necesarias para React Navigation y sus complementos.
- Configurar el archivo babel.config.js para soportar animaciones y otras optimizaciones.
- Importar y configurar cada tipo de navegación con sus respectivas pantallas.

Ejemplo Integrador

- Se muestra cómo combinar los tres tipos de navegación en una sola app:
- Un Drawer Navigator como menú principal.
- Dentro del Drawer, un Stack Navigator con pantallas Home y Details.
- También un Tab Navigator con pestañas para Settings y Profile.
- Cada pantalla incluye componentes básicos y botones para navegar entre pantallas.



SABERES
PREVIOS



¿Cómo se comunican las aplicaciones en los sistemas distribuidos?

Al finalizar este bloque el alumno explica el manejo de APIs, fetch y axios en el desarrollo de una app móvil con React Native, mediante el ejemplos de código que consuman y renderizen datos de una API pública.



Concepto de API REST y uso de fetch

- Una API REST es una interfaz que sigue los principios de diseño REST (Transferencia de Estado Representacional), usada para conectar sistemas distribuidos. Permite que una aplicación cliente acceda y manipule recursos (datos) en un servidor mediante métodos HTTP (GET, POST, PUT, DELETE). Es ligera, escalable y común para el intercambio de datos entre aplicaciones y servicios web.

Uso de fetch en React Native

Fetch es un método nativo para hacer solicitudes HTTP. En React Native se usa dentro de hooks como useEffect para realizar peticiones y recibir datos asíncronos, gestionando estados de carga y error. Ejemplo básico para obtener datos de API pública JSONPlaceholder:

El código en la imagen maneja estados para el fetch, muestra un indicador de carga y renderiza títulos de posts.

```
jsx
import React, { useState, useEffect } from 'react';
import { View, Text, FlatList } from 'react-native';

export default function App() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts?_limit=10')
      .then(response => response.json())
      .then(data => {
        setPosts(data);
        setLoading(false);
      })
      .catch(error => {
        console.error(error);
        setLoading(false);
      });
  }, []);

  if (loading) return <Text>Cargando datos...</Text>;

  return (
    <View>
      <FlatList
        data={posts}
        keyExtractor={item => item.id.toString()}
        renderItem={({ item }) => (
          <Text>{item.title}</Text>
        )}
      />
    </View>
  );
}
```

Uso de Axios en React Native

Ejemplo en React Native consumiendo el mismo endpoint de la lámina anterior:

Axios es una librería popular para hacer solicitudes HTTP. Simplifica las llamadas al soportar promesas, transformación automática JSON, manejo de errores, y configuración centralizada.

Axios permite configurar una instancia con baseURL para simplificar futuras llamadas

Instalar la dependencia: `npm install axios`

```
jsx

import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { View, Text, FlatList } from 'react-native';

export default function App() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/posts?_limit=10')
      .then(response => {
        setPosts(response.data);
        setLoading(false);
      })
      .catch(error => {
        console.error(error);
        setLoading(false);
      });
  }, []);

  if (loading) return <Text>Cargando datos...</Text>;

  return (
    <View>
      <FlatList
        data={posts}
        keyExtractor={item => item.id.toString()}
        renderItem={({ item }) => (
          <Text>{item.title}</Text>
        )}
      />
    </View>
  );
}
```


Aplicación móvil consumiendo API pública

Aquí está un proyecto completo listo para ejecutar en React Native con Expo, que consume datos de una API pública usando fetch y renderiza los datos con FlatList. Este proyecto incluye manejo de estados de carga y errores

Instalar la dependencia:

```
npx expo install react-native-safe-area-context
```

```
1 import React, { useState, useEffect } from 'react';
2 import { View, Text, FlatList, ActivityIndicator, StyleSheet } from 'react-native';
3 import { SafeAreaView } from 'react-native-safe-area-context';
4 export default function App() {
5   const [data, setData] = useState([]);
6   const [loading, setLoading] = useState(true);
7   const [error, setError] = useState(null);
8   useEffect(() => {
9     const fetchData = async () => {
10       try {
11         const response = await fetch('https://jsonplaceholder.typicode.com/posts?_limit=10');
12         if (!response.ok) {
13           throw new Error(`Error HTTP: ${response.status}`);
14         }
15         const json = await response.json();
16         setData(json);
17       } catch (error) {
18         setError(error.message);
19       } finally {
20         setLoading(false);
21       }
22     };
23     fetchData();
24   }, []);
25   if (loading) {
26     return (
27       <SafeAreaView style={styles.centered}>
28         <ActivityIndicator size="large" color="#0000ff" />
29         <Text>Cargando datos...</Text>
30       </SafeAreaView>
31     );
32   }
```

```
33   if (error) {
34     return (
35       <SafeAreaView style={styles.centered}>
36         <Text style={styles.errorText}>Error: {error}</Text>
37       </SafeAreaView>
38     );
39   }
40   return (
41     <SafeAreaView style={styles.container}>
42       <Text style={styles.header}>Posts desde API pública</Text>
43       <FlatList
44         data={data}
45         keyExtractor={({ item }) => item.id.toString()}
46         renderItem={({ item }) => (
47           <View style={styles.item}>
48             <Text style={styles.title}>{item.title}</Text>
49             <Text>{item.body}</Text>
50           </View>
51         )}
52       />
53     </SafeAreaView>
54   );
55 }
56 const styles = StyleSheet.create({
57   container: { flex: 1, padding: 20, backgroundColor: '#f0f0f0' },
58   centered: { flex: 1, justifyContent: 'center', alignItems: 'center' },
59   header: { fontSize: 24, fontWeight: 'bold', marginBottom: 15 },
60   item: { backgroundColor: 'white', padding: 15, marginBottom: 10, borderRadius: 8 },
61   title: { fontWeight: 'bold', marginBottom: 5 },
62   errorText: { color: 'red' },
63 });
```



Aplicación móvil consumiendo API pública

Este código utiliza axios para hacer la petición GET a la API pública, con manejo de parámetros usando params, controla estados de carga y error, y renderiza los datos con FlatList en React Native de forma equivalente al ejemplo con fetch

Instalar la dependencia:

```
npx expo install react-native-safe-area-context
```

```
npm install axios
```

```
1  import React, { useState, useEffect } from 'react';
2  import { View, Text, FlatList, ActivityIndicator, StyleSheet } from 'react-native';
3  import { SafeAreaView } from 'react-native-safe-area-context';
4  import axios from 'axios';
5  export default function App() {
6    const [data, setData] = useState([]);
7    const [loading, setLoading] = useState(true);
8    const [error, setError] = useState(null);
9    useEffect(() => {
10      const fetchData = async () => {
11        try {
12          const response = await axios.get('https://jsonplaceholder.typicode.com/posts', {
13            params: { _limit: 10 }
14          });
15          setData(response.data);
16        } catch (error) {
17          setError(error.message);
18        } finally {
19          setLoading(false);
20        }
21      };
22      fetchData();
23    }, []);
24    if (loading) {
25      return (
26        <SafeAreaView style={styles.centered}>
27          <ActivityIndicator size="large" color="#0000ff" />
28          <Text>Cargando datos...</Text>
29        </SafeAreaView>
30      );
31    }
```

```
32    if (error) {
33      return (
34        <SafeAreaView style={styles.centered}>
35          <Text style={styles.errorText}>Error: {error}</Text>
36        </SafeAreaView>
37      );
38    }
39    return (
40      <SafeAreaView style={styles.container}>
41        <Text style={styles.header}>Posts desde API pública</Text>
42        <FlatList
43          data={data}
44          keyExtractor={({item}) => item.id.toString()}
45          renderItem={({item}) => (
46            <View style={styles.item}>
47              <Text style={styles.title}>{item.title}</Text>
48              <Text>{item.body}</Text>
49            </View>
50          )}
51        />
52      </SafeAreaView>
53    );
54  }
55  const styles = StyleSheet.create({
56    container: { flex: 1, padding: 20, backgroundColor: '#f0f0f0' },
57    centered: { flex: 1, justifyContent: 'center', alignItems: 'center' },
58    header: { fontSize: 24, fontWeight: 'bold', marginBottom: 15 },
59    item: { backgroundColor: 'white', padding: 15, marginBottom: 10, borderRadius: 8 },
60    title: { fontWeight: 'bold', marginBottom: 5 },
61    errorText: { color: 'red' },
62  });
```



Ejemplo con fetch, async/await y FlatList

Este ejemplo carga 14 posts desde la API pública JSONPlaceholder, mostrará indicador de carga y control de errores, y renderiza la lista con FlatList en React Native usando fetch con async/await.

Instalar la dependencia:

```
npx expo install react-native-safe-area-context
```

```
1 import { SafeAreaView } from 'react-native-safe-area-context';
2 import React, { useState, useEffect } from 'react';
3 import { View, Text, FlatList, ActivityIndicator, StyleSheet } from 'react-native';
4 export default function App() {
5   const [data, setData] = useState([]);
6   const [loading, setLoading] = useState(true);
7   const [error, setError] = useState(null);
8   useEffect(() => {
9     const fetchData = async () => {
10       try {
11         const response = await fetch('https://jsonplaceholder.typicode.com/posts?_limit=14');
12         if (!response.ok) {
13           throw new Error(`HTTP error! status: ${response.status}`);
14         }
15         const json = await response.json();
16         setData(json);
17       } catch (error) {
18         setError(error.message);
19       } finally {
20         setLoading(false);
21       }
22     };
23     fetchData();
24   }, []);
25   if (loading) {
26     return (
27       <SafeAreaView style={styles.centered}>
28         <ActivityIndicator size="large" color="#0000ff" />
29         <Text>Cargando datos...</Text>
30       </SafeAreaView>
31     );
32   }
```

```
33   if (error) {
34     return (
35       <SafeAreaView style={styles.centered}>
36         <Text style={styles.error}>Error: {error}</Text>
37       </SafeAreaView>
38     );
39   }
40   return (
41     <SafeAreaView style={styles.container}>
42       <Text style={styles.header}>Lista de Posts</Text>
43       <FlatList
44         data={data}
45         keyExtractor={({ item }) => item.id.toString()}
46         renderItem={({ item }) => (
47           <View style={styles.item}>
48             <Text style={styles.title}>{item.title}</Text>
49             <Text>{item.body}</Text>
50           </View>
51         )}
52       />
53     </SafeAreaView>
54   );
55 }
56 const styles = StyleSheet.create({
57   container: { flex: 1, padding: 20, backgroundColor: '#f9f9f9' },
58   centered: { flex: 1, justifyContent: 'center', alignItems: 'center' },
59   header: { fontSize: 24, fontWeight: 'bold', marginBottom: 15 },
60   item: { backgroundColor: 'white', padding: 15, marginVertical: 8, borderRadius: 8, elevation: 2 },
61   title: { fontWeight: 'bold', fontSize: 16, marginBottom: 5 },
62   error: { color: 'red' },
63 });
```



Ejemplo con axios, async/await y FlatList

Este código usa Axios para hacer la petición GET a la misma API pública con límite 14, maneja estados de carga, error y datos, y renderiza la lista con FlatList y estilos iguales al código original con fetch

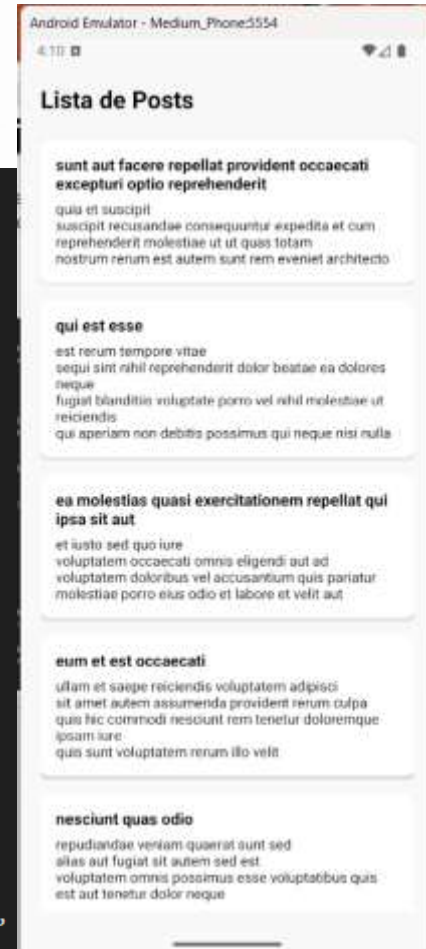
Instalar la dependencia:

```
npx expo install react-native-safe-area-context
```

```
npm install axios
```

```
1 import { SafeAreaView } from 'react-native-safe-area-context';
2 import React, { useState, useEffect } from 'react';
3 import { View, Text, FlatList, ActivityIndicator, StyleSheet } from 'react-native';
4 import axios from 'axios';
5 export default function App() {
6   const [data, setData] = useState([]);
7   const [loading, setLoading] = useState(true);
8   const [error, setError] = useState(null);
9   useEffect(() => {
10     const fetchData = async () => {
11       try {
12         const response = await axios.get('https://jsonplaceholder.typicode.com/posts', {
13           params: { _limit: 14 }
14         });
15         setData(response.data);
16       } catch (error) {
17         setError(error.message);
18       } finally {
19         setLoading(false);
20       }
21     };
22     fetchData();
23   }, []);
24   if (loading) {
25     return (
26       <SafeAreaView style={styles.centered}>
27         <ActivityIndicator size="large" color="#0000ff" />
28         <Text>Cargando datos...</Text>
29       </SafeAreaView>
30     );
31   }
```

```
32   if (error) {
33     return (
34       <SafeAreaView style={styles.centered}>
35         <Text style={styles.error}>Error: {error}</Text>
36       </SafeAreaView>
37     );
38   }
39   return (
40     <SafeAreaView style={styles.container}>
41       <Text style={styles.header}>Lista de Posts</Text>
42       <FlatList
43         data={data}
44         keyExtractor={({ item }) => item.id.toString()}
45         renderItem={({ item }) => (
46           <View style={styles.item}>
47             <Text style={styles.title}>{item.title}</Text>
48             <Text>{item.body}</Text>
49           </View>
50         )}
51       />
52     </SafeAreaView>
53   );
54 }
55 const styles = StyleSheet.create({
56   container: { flex: 1, padding: 20, backgroundColor: '#f9f9f9' },
57   centered: { flex: 1, justifyContent: 'center', alignItems: 'center' },
58   header: { fontSize: 24, fontWeight: 'bold', marginBottom: 15 },
59   item: { backgroundColor: 'white', padding: 15, marginVertical: 8, borderRadius: 8, elevation: 2 },
60   title: { fontWeight: 'bold', fontSize: 16, marginBottom: 5 },
61   error: { color: 'red' },
62 });
```



Añadir paginación infinita a la FlatList

Ejemplo práctico de cómo añadir paginación infinita (scroll infinito) a una FlatList en React Native usando fetch, async/await y estado para manejar páginas y carga de datos:

Instalar la dependencia:
`npx expo install react-native-safe-area-context`

```
1 import { SafeAreaView } from 'react-native-safe-area-context';
2 import React, { useState, useEffect } from 'react';
3 import { View, Text, FlatList, ActivityIndicator, StyleSheet } from 'react-native';
4 export default function App() {
5   const [data, setData] = useState([]);
6   const [page, setPage] = useState(1);
7   const [loading, setLoading] = useState(false);
8   const [error, setError] = useState(null);
9   const [refreshing, setRefreshing] = useState(false);
10  const [hasMore, setHasMore] = useState(true);
11  const fetchData = async (pageNum = 1, refreshing = false) => {
12    if (loading) return; // evitar llamadas simultáneas
13    setLoading(true);
14    setError(null);
15    try {
16      const response = await fetch(`https://jsonplaceholder.typicode.com/posts?_limit=10&_page=${pageNum}`);
17      if (!response.ok) {
18        throw new Error(`HTTP error! status: ${response.status}`);
19      }
20      const json = await response.json();
21      if (refreshing) {
22        setData(json);
23      } else {
24        setData(prevData => [...prevData, ...json]);
25      }
26      // Si la respuesta es menor que el límite, no hay más datos
27      setHasMore(json.length === 10);
28    } catch (err) {
29      setError(err.message);
30    } finally {
31      setLoading(false);
32      if (refreshing) setRefreshing(false);
33    }
34  };
35  useEffect(() => {
36    fetchData(page);
37  }, [page]);
38  const handleLoadMore = () => {
39    if (hasMore && !loading) {
40      setPage(prevPage => prevPage + 1);
41    }
42  };
43  const handleRefresh = () => {
44    setRefreshing(true);
45    setPage(1);
46    fetchData(1, true);
47  };
48  const renderFooter = () => {
49    if (!loading) return null;
50    return (
51      <View style={styles.footer}>
52        <ActivityIndicator size="large" />
53      </View>
54    );
55  };
56  if (error) {
57    return (
58      <SafeAreaView style={styles.centered}>
59        <Text style={styles.error}>Error: {error}</Text>
60      </SafeAreaView>
61    );
62  }
63  return (
64    <SafeAreaView style={styles.container}>
65      <Text style={styles.header}>Posts con paginación infinita</Text>
66      <FlatList
67        data={data}
68        keyExtractor={item => item.id.toString()}
69        renderItem={({ item }) => (
70          <View style={styles.item}>
71            <Text style={styles.title}>{item.title}</Text>
72            <Text>{item.body}</Text>
73          </View>
74        )}
75        onEndReached={handleLoadMore}
76        onEndReachedThreshold={0.5}
77        ListFooterComponent={renderFooter}
78        refreshing={refreshing}
79        onRefresh={handleRefresh}
80      />
81    </SafeAreaView>
82  );
83 }
84 const styles = StyleSheet.create({
85   container: { flex: 1, padding: 20, backgroundColor: '#f9f9f9' },
86   header: { fontSize: 24, fontWeight: 'bold', marginBottom: 15 },
87   item: { backgroundColor: '#fff', padding: 15, marginVertical: 8, borderRadius: 8, elevation: 2 },
88   title: { fontWeight: 'bold', fontSize: 16, marginBottom: 5 },
89   error: { color: 'red', alignSelf: 'center', marginTop: 20 },
90   footer: { paddingVertical: 20 },
91   centered: { flex: 1, justifyContent: 'center', alignItems: 'center' },
92 });
```

Explicación:

- Se mantiene un estado `page` para controlar la página actual.
- `fetchData` realiza la petición con el parámetro `_page` y concatena los datos recibidos.
- `onEndReached` llama a `handleLoadMore` para incrementar la página, disparando la carga de más datos.
- `refreshing` y `onRefresh` permiten hacer pull-to-refresh para recargar desde la primera página.
- Controla si quedan más datos con `hasMore`, para evitar cargar innecesariamente.
- Se muestra un indicador de carga en el pie de lista mientras se cargan más datos.

Este patrón es común para listas con paginación infinita en aplicaciones móviles React Native.

```
63 return (
64   <SafeAreaView style={styles.container}>
65     <Text style={styles.header}>Posts con paginación infinita</Text>
66     <FlatList
67       data={data}
68       keyExtractor={item => item.id.toString()}
69       renderItem={({ item }) => (
70         <View style={styles.item}>
71           <Text style={styles.title}>{item.title}</Text>
72           <Text>{item.body}</Text>
73         </View>
74       )}
75       onEndReached={handleLoadMore}
76       onEndReachedThreshold={0.5}
77       ListFooterComponent={renderFooter}
78       refreshing={refreshing}
79       onRefresh={handleRefresh}
80     />
81   </SafeAreaView>
82 );
83 }
84 const styles = StyleSheet.create({
85   container: { flex: 1, padding: 20, backgroundColor: '#f9f9f9' },
86   header: { fontSize: 24, fontWeight: 'bold', marginBottom: 15 },
87   item: { backgroundColor: '#fff', padding: 15, marginVertical: 8, borderRadius: 8, elevation: 2 },
88   title: { fontWeight: 'bold', fontSize: 16, marginBottom: 5 },
89   error: { color: 'red', alignSelf: 'center', marginTop: 20 },
90   footer: { paddingVertical: 20 },
91   centered: { flex: 1, justifyContent: 'center', alignItems: 'center' },
92 });
```



Uso de un archivo JSON en una App móvil

```
1 import { StatusBar } from 'expo-status-bar';
2 import { StyleSheet, Text, View, ScrollView, Image } from 'react-native';
3 import { SafeAreaView } from 'react-native-safe-area-context';
4 import data from './data/women.json';
5 console.log(data);
6 export default function App() {
7   return (
8     <SafeAreaView style={{ flex: 1 }}>
9       <View style={styles.container}>
10        <Text>Desarrollo de Aplicaciones Móviles</Text>
11        <ScrollView>
12          {data.map((producto) => {
13            return (
14              <View key={producto.id}>
15                <Text>{producto.name}</Text>
16                <Text>{producto.nationality}</Text>
17                <Image
18                  source={{uri: producto.photo}}
19                  style={styles.imageSize}
20                />
21              </View>);
22            })}
23        </ScrollView>
24        <StatusBar style="auto" />
25      </View>
26    </SafeAreaView>
27  );
28 }
29 const styles = StyleSheet.create({
30   container: {
31     flex: 1,
32     backgroundColor: 'white',
33     alignItems: 'center',
34     justifyContent: 'center',
35   },
36   imageSize: {
37     width: 100,
38     height: 100,
39   }
40 });
```



Uso de un archivo JSON con FlatList

```
1 import { StatusBar } from 'expo-status-bar';
2 import { StyleSheet, Text, View, ScrollView, Image, FlatList } from 'react-native';
3 import { SafeAreaView } from 'react-native-safe-area-context';
4 import data from './data/women.json';
5 console.log(data);
6 export default function App() {
7   return (
8     <SafeAreaView style={{ flex: 1 }}>
9       <View style={styles.container}>
10        <Text>Desarrollo de Aplicaciones Móviles</Text>
11        <FlatList
12          data={data}
13          keyExtractor={item => item.id}
14          renderItem={({ item }) =>
15            <View key={item.id}>
16              <Text>{item.name}</Text>
17              <Text>{item.nationality}</Text>
18              <Image
19                source={{ uri: item.photo }}
20                style={styles.imageSize}
21              />
18              </View>
22            </View>
23          )
24        />
25        <StatusBar style="auto" />
26      </View>
27    </SafeAreaView>
28  );
29 }
30 const styles = StyleSheet.create({
31   container: {
32     flex: 1,
33     backgroundColor: 'white',
34     alignItems: 'center',
35     justifyContent: 'center',
36   },
37   imageSize: {
38     width: 100,
39     height: 100,
40   }
41 });
```



Uso de una API en una App móvil

```
1 import React, { useState } from 'react';
2 import { View, Text, TextInput, Button, FlatList, Image, StyleSheet, ActivityIndicator } from 'react-native';
3 export default function ProductListScreen() {
4   const [query, setQuery] = useState('natural');
5   const [products, setProducts] = useState([]);
6   const [loading, setLoading] = useState(false);
7   const fetchProducts = async () => {
8     setLoading(true);
9     try {
10       const res = await fetch(
11         'https://world.openfoodfacts.org/cgi/search.pl?search_terms=${encodeURIComponent(query)}&search_simple=1&action-process&json=1&page=1&page_size=30'
12       );
13       const data = await res.json();
14       const prods = data.products.map((p) => ({
15         id: p.code,
16         nombre: p.product_name || 'Sin nombre',
17         caracteristicas: p.ingredients_text || 'Sin descripción',
18         imagen: p.image_url || null,
19       }));
20       setProducts(prods);
21     } catch (error) {
22       console.error('Error fetch:', error);
23     }
24     setLoading(false);
25   };
26   return (
27     <View style={styles.container}>
28       <Text style={styles.header}>Productos Naturales</Text>
29       <View style={styles.searchContainer}>
30         <TextInput
31           placeholder="Buscar productos..."
32           value={query}
33           onChangeText={setQuery}
34           style={styles.input}
35         />
36         <Button title="Buscar" onPress={fetchProducts} />
37       </View>
38       <View style={styles.products}>
39         {loading ? (
40           <ActivityIndicator size="large" color="#0000ff" />
41         ) : (
42           <FlatList
43             data={products}
44             keyExtractor={(item) => item.id}
45             renderItem={({ item }) => (
46               <View style={styles.item}>
47                 {item.imagen ? (
48                   <Image source={{ uri: item.imagen }} style={styles.image} />
49                 ) : (
50                   <View style={styles.noImage}>
51                     <Text>No Image</Text>
52                   </View>
53                 )}
54                 <View style={styles.info}>
55                   <Text style={styles.name}>{item.nombre}</Text>
56                   <Text style={styles.features} numberOfLines={3}>
57                     {item.caracteristicas}
58                   </Text>
59                 </View>
60               </View>
61             )}
62             contentContainerStyle={{ paddingBottom: 20 }}
63           />
64         )}
65       </View>
66     </View>
67   );
68 }
69
70 const styles = StyleSheet.create({
71   container: { flex: 1, padding: 10, backgroundColor: '#f7f7f7' },
72   header: { fontSize: 24, fontWeight: 'bold', marginBottom: 10 },
73   searchContainer: { flexDirection: 'row', margin: 10 },
74   input: { flex: 1, borderWidth: 1, borderColor: '#ccc', backgroundColor: '#fff', borderRadius: 4, padding: 10 },
75   item: { flex: 1, margin: 10, padding: 10, borderRadius: 6, elevation: 1 },
76   image: { width: 80, height: 80, borderRadius: 6 },
77   noImage: { width: 80, height: 80, borderRadius: 6, backgroundColor: '#e3e3e3', display: flex, alignContent: 'center', justify-content: center },
78   info: { flex: 1, margin: 10 },
79   name: { fontWeight: 'bold', fontSize: 16, margin: 0 },
80   features: { fontSize: 14, color: '#555', margin: 0 }
81 });
```

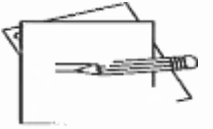


Implementación de la app móvil con una interfaz básica



¿Preguntas?





Concepto de API REST

- Una API REST es una interfaz que sigue los principios REST (Transferencia de Estado Representacional).
- Permite a clientes acceder y manipular recursos en servidor vía métodos HTTP (GET, POST, PUT, DELETE).
- Es ligera, escalable y estándar en la comunicación entre aplicaciones y servicios web.

Uso de fetch en React Native

- fetch es un método nativo para solicitudes HTTP.
- Se usa dentro de hooks como useEffect para llamadas asíncronas.
- Incluye manejo de estados de carga y error.
- Ejemplo básico obtiene datos de la API pública JSONPlaceholder, mostrando lista de títulos con FlatList.
- Se demuestra cómo manejar promesas y actualizar estado en React.

Uso de Axios en React Native

- Axios es una librería popular que facilita llamadas HTTP.
- Simplifica el manejo de promesas, transforma JSON automáticamente, y permite configuración central.
- Se puede crear una instancia para simplificar llamadas repetidas con baseURL fija.
- Ejemplo equivalente al fetch, consume la misma API y renderiza resultados con FlatList. Requiere instalar la dependencia axios para su uso.

Ejemplos mostrados

- Ejemplos que consumen API pública con fetch o Axios, muestra datos en FlatList. Gestiona estados de carga (ActivityIndicator) y errores (mensaje en pantalla).
- Paginación infinita con FlatList (scroll infinito), paginación dinámica usando FlatList. Mantiene estado para página actual, carga y si hay más datos. Llama a la API con parámetros de página y concatena resultados. Incluye manejo de pull-to-refresh para recargar la lista. Muestra indicador de carga en pie de lista mientras se cargan más datos. Patrón común para implementar listas largas eficientes en dispositivos móviles.

FIN



Presentación del Avance del proyecto?